Day 1 Study Material

Month 1: Core Foundations & Vanilla TypeScript | Topic: How JavaScript Runs (Execution Context & Call Stack)

Time needed: about 3 to 4 hours **Rule for this month:** type all code yourself, no AI code generation.

Today's Goal

By the end of today you should be able to explain, in your own simple words, **what happens inside JavaScript when your code runs**. This is the base for closures, promises and async/await later.

Step-by-Step Plan

Step	What to do	Time
1	Set up tools (Node.js, VS Code, Git, GitHub repo)	30 min
2	Watch Namaste JavaScript, first 2 episodes	60 min
3	Read the notes below and understand each idea	30 min
4	Run the practice code and draw the call stack on paper	45 min
5	Do the exercises and the self-check	30 min
6	Commit your notes to GitHub	10 min

Step 1: Setup

- Install **Node.js** (LTS version) from nodejs.org. Check it: open terminal and type `node -v`.
- Install **VS Code**. Install **Git** and check it with `git --version`.
- On GitHub, make a new repo named `fullstack-roadmap`. Clone it to your computer.
- Inside it, make a folder `month-1/day-1`. All of today's files go here.

Step 2: Watch the Lecture

- **Namaste JavaScript by Akshay Saini** (from your roadmap): https://www.youtube.com/watch?v=pN6jk0uUrD8
- Watch the **first two episodes** (how JavaScript works, and how JavaScript code is executed with the call stack).
- Pause often. Write short notes in your own words. Do not just copy what the speaker says.

Step 3: Easy Notes

Idea 1: Everything runs inside an Execution Context

Think of an **execution context** as a big box where your code runs. When your program starts, JavaScript makes one box called the **Global Execution Context**. Each time you call a function, JavaScript makes a new small box for that function.

Idea 2: Each box has two phases

- **Phase 1: Memory phase.** JavaScript reads the code from top to bottom and sets aside memory for every variable and function. Variables get the value `undefined`. Functions are stored fully.

- **Phase 2: Code execution phase.** JavaScript now runs the code line by line, one line at a time, and puts real values in the variables.

Idea 3: JavaScript is single-threaded

It can do **only one thing at a time**, in order. It finishes one line before it moves to the next.

Idea 4: The Call Stack

The **call stack** is a pile of execution contexts. Think of a stack of plates: you add a plate on top, and you can only remove the top plate. When a function is called, its context goes on top. When the function finishes, its context is removed. When the stack is empty, the program is done.

Step 4: Practice Code

Create a file `day1.js`. Type this code yourself (do not paste):

```
var n = 2;

function square(num) {
  var ans = num * num;
  return ans;
}

var square2 = square(n);
var square4 = square(4);

console.log(square2);
console.log(square4);
```

Run it with `node day1.js`. You should see 4 and 16. Now understand **why**:

- **Memory phase (global):** n = undefined, square = (the whole function), square2 = undefined, square4 = undefined.
- **Execution phase:** n becomes 2. Then `square(n)` is called, so a new context is made and put on top of the call stack.
- Inside that context: num = 2 and ans = undefined (memory phase), then ans = 4 (execution phase). The function returns 4 and its context is removed from the stack.
- square2 now holds 4. The same thing happens again for `square(4)`.

Hoisting: see it yourself

Hoisting is just the result of the memory phase. Type and run this:

```
console.log(a);   // what do you think this prints?
sayHi();          // does this work before it is written?

var a = 10;

function sayHi() {
  console.log("Hi!");
}
```

It prints `undefined` and then Hi!. Why? Because in the memory phase, a is set to undefined and `sayHi` is stored completely, before any line runs.

Call stack with nested functions

```
function one() {
  console.log("one start");
```

```
  two();
  console.log("one end");
}
function two() {
  console.log("two start");
  three();
  console.log("two end");
}
function three() {
  console.log("three");
}
one();
```

Draw the stack on paper. Order of the stack from bottom to top: **global, one, two, three**. Then they leave in reverse: three, two, one. Output order: one start, two start, three, two end, one end.

Step 5: Exercises

- **1.** In your own words, write 3 lines: what is an execution context?
- **2.** Change the nested-functions example so `three()` also calls a new function `four()`. Predict the output first, then run it.
- **3.** Write a small program with 2 variables and 2 functions. On paper, write the memory phase and then the execution phase.
- **4.** In the browser, open DevTools, go to Sources, put a breakpoint inside `three()`, and look at the **Call Stack** panel on the right.
- **5.** Make a mistake on purpose: write a function that calls itself forever. Read the error message (Maximum call stack size exceeded) and explain why it happens.

Self-Check: Can You Answer These?

- What are the two phases of an execution context?
- Why does a variable print `undefined` when used before its line?
- What does it mean that JavaScript is single-threaded?
- What happens to a function's context when the function finishes?

If you can answer all four in simple words without looking, Day 1 is done.

Step 6: Save Your Work

```
git add .
git commit -m "Day 1: execution context and call stack"
git push
```

Tomorrow (Day 2) Preview

Hoisting in more detail (var, let, const), the temporal dead zone, and scope. This builds directly on today's memory phase idea.